Latest updates: https://dl.acm.org/doi/10.1145/3719027.3765046

RESEARCH-ARTICLE

SISTAR: An Efficient DDoS Detection and Mitigation Framework Utilizing Programmable Data Planes

JUNJIE HU, Institute of Information Engineering, Beijing, China

FENG GUO, Institute of Information Engineering, Beijing, China

QIHANG ZHOU, Institute of Information Engineering, Beijing, China

YIXIN ZHANG, Institute of Information Engineering, Beijing, China

ZIBO GAO, Institute of Information Engineering, Beijing, China

YINGLONG HAN, Institute of Information Engineering, Beijing, China

View all

Open Access Support provided by:

Institute of Information Engineering

PDF Download
3719027.3765046.pdf
07 April 2026
Total Citations: 0
Total Downloads: 2574

Published: 22 November 2025

Citation in BibTeX format

CCS '25: ACM SIGSAC Conference on Computer and Communications Security
October 13 - 17, 2025
Taipei, Taiwan

Conference Sponsors:
SIGSAC

CCS '25: Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (November 2025)
https://doi.org/10.1145/3719027.3765046
ISBN: 9798400715259

SISTAR: An Efficient DDoS Detection and Mitigation Framework Utilizing Programmable Data Planes

Junjie Hu
Institute of Information Engineering, Chinese Academy of Sciences
School of Cyber Security, University of Chinese Academy of Sciences
Beijing, China
hujunjie@iie.ac.cn

Feng Guo
Institute of Information Engineering, Chinese Academy of Sciences
School of Cyber Security, University of Chinese Academy of Sciences
Beijing, China
guofeng@iie.ac.cn

Qihang Zhou
Institute of Information Engineering, Chinese Academy of Sciences
School of Cyber Security, University of Chinese Academy of Sciences
Beijing, China
zhouqihang@iie.ac.cn

Yixin Zhang
Institute of Information Engineering, Chinese Academy of Sciences
School of Cyber Security, University of Chinese Academy of Sciences
Beijing, China
zhangyixin@iie.ac.cn

Zibo Gao
Institute of Information Engineering, Chinese Academy of Sciences
School of Cyber Security, University of Chinese Academy of Sciences
Beijing, China
gaozibo@iie.ac.cn@iie.ac.cn

Yinglong Han
Institute of Information Engineering, Chinese Academy of Sciences
School of Cyber Security, University of Chinese Academy of Sciences
Beijing, China
hanyinglong@iie.ac.cn

Zhiqiang Lv*
Institute of Information Engineering, Chinese Academy of Sciences
School of Cyber Security, University of Chinese Academy of Sciences
Beijing, China
lvzhiqiang@iie.ac.cn

Abstract

DDoS attacks have become one of the most severe cybersecurity threats, especially in application-layer attacks. With the emergence of Programmable Data Planes (PDPs), it has become possible to maintain line-rate throughput while achieving high detection rates, making them crucial in addressing DDoS challenges. However, due to the complexity of DDoS attacks, detection remains resource-intensive and overall network defense effectiveness is limited. This limitation becomes particularly pronounced in clustered environments, where coordinated defense is essential. This paper presents SISTAR, an innovative framework for efficient DDoS detection and mitigation using PDP. SISTAR integrates an improved Decision Tree - Constrained Threshold Segmentation (DT-CTS) model to achieve high detection accuracy while minimizing hardware resource usage. Through distributed deployment across multiple switches, SISTAR enhances network resilience by enabling rapid detection and coordinated response to DDoS attacks. We implement a prototype of SISTAR and evaluate its performance in a realistic testbed, the experimental results show that SISTAR surpasses existing models in terms of detection accuracy and resource efficiency. When combined with its alert pushback mechanism, SISTAR can effectively reduce network resource consumption caused by DDoS attacks.

CCS Concepts

• **Security and privacy → Denial-of-service attacks**.

Keywords

Programmable Data Plane, Decision Tree, DDoS Detection and Mitigation

ACM Reference Format:
Junjie Hu, Feng Guo, Qihang Zhou, Yixin Zhang, Zibo Gao, Yinglong Han, and Zhiqiang Lv. 2025. SISTAR: An Efficient DDoS Detection and Mitigation Framework Utilizing Programmable Data Planes. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25), October 13–17, 2025, Taipei.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3719027.3765046

*Corresponding author.

This work is licensed under a Creative Commons Attribution 4.0 International License.
CCS '25, Taipei.
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1525-9/2025/10
https://doi.org/10.1145/3719027.3765046

1 Introduction

Distributed Denial of Service (DDoS) attacks are among the most pervasive threats in modern network environments [19, 28, 47]. These attacks overwhelm target servers, services, or network resources with floods of illegitimate requests, rendering them incapable of serving legitimate users. The expansion of Internet of Things (IoT) devices has increased the frequency and scale

of DDoS attacks, complicating the efforts to mount effective defenses [4, 13, 71]. Notably, alongside traditional volumetric attacks [2, 58], there is a rising prevalence of application-layer and protocol-based attacks [36, 62], further intensifying the challenge.

The emergence of Programmable Data Planes (PDP) enables DDoS detection directly on switches [5, 35]. Unlike traditional detection methods, the PDP can maintain the flow connection state, obtain fine-grained information directly from packets, and collect more accurate network traffic statistical data [6, 33]. In-network computation using programmable switch ASICs offers significant advantages over specialized hardware (expensive, limited functionality) and software platforms (high processing delays) [72], enabling real-time traffic inspection at line rate with low latency while scaling to the network edge. Many methods have deployed decision tree (DT) and other machine learning models on PDP to detect traffic for defense against various malicious threats. However, these methods face significant challenges in practical applications.

First, there is a lack of comprehensive consideration of flow characteristics. For example, Mousika [64] achieves deployment by modifying the DT to binary decision tree, but only extracts the features of a single packet without considering the dynamic changes of flows. pForest [8] and SwitchTree [27] can handle flow features and perform real-time classification, but their random forest (RF) models deployed due to resource limitations suffer from reduced depth and accuracy, limiting their application in complex network environments. **Second, there is significant performance and hardware overhead.** For example, the NetBeacon [77], IIsy [75], and Planter [76] frameworks generate efficient machine learning models for application on programmable switches, but require too many features, consuming huge table entries on switches, and even requiring hundreds of flow table entries for high-precision classification. Although Soter [63] adopts a two-stage detection method, first performing initial detection on programmable switches and then sending suspicious traffic to the CPU for thorough detection, it still faces bottlenecks under high load. **Third, there is a lack of effective strategies to counter application-layer DDoS attacks.** Poseidon [72] and Jaqen [31] rely solely on protocol type judgments, making it difficult to update defense strategies promptly in response to evolving application-layer attacks. Furthermore, this approach may introduce new vulnerabilities, especially targeting controllers. **Finally, these methods do not consider the linkage protection of switch clusters.** Especially in the flooding and application-layer DDoS attack scenarios, the existing protection methods often fail to consider the deployment of programmable switches such as Ali's Sailfish cloud gateway [44] and large-scale programmable switches supporting cloud services, which are often easy to become attack targets.

Therefore, it is necessary to propose a high-performance framework for detecting and mitigating flooding and application-layer DDoS attacks in multi-switch scenarios, which considers the characteristics of flows. To solve these problems, our overall design goal is to reduce the occupation of hardware resources of switches as much as possible while maintaining high-precision detection of DDoS attacks, and at the same time avoiding the controllers' participation as much as possible. Upon detecting an attack, a switch should proactively notify other switches of the attacker's information to enable collaborative network-wide defense, rather than isolated local protection.

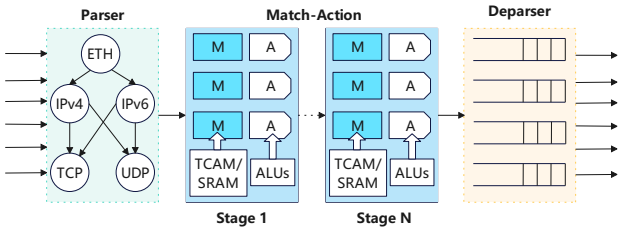


Figure 1: Programmable Data Plane. The data plane consists of a parser, a deparser, and a series of match-action pipelines.

Based on the above claims, this paper proposes SISTAR[1], an efficient DDoS attack detection and mitigation framework for PDP, aimed at addressing the shortcomings of existing methods. For a wide range of protocol-based and application-layer DDoS attacks, SISTAR combines packet-level and flow-level features with low correlation. To address resource consumption issues, we optimized the machine learning model through threshold pruning and deployed it distributively on programmable switches via feature encoding. Coupled with a designed alert information transmission mechanism, SISTAR can accurately detect and execute pushback operations during DDoS attacks, thereby minimizing network resource usage.

We implemented a prototype of SISTAR using a Tofino hardware switch and BMv2 software switch for simulation, and evaluated SISTAR by replaying representative DDoS traffic to assess its detection and mitigation capabilities. Experimental results demonstrate that SISTAR surpasses existing models in terms of DDoS attack detection accuracy and hardware resource performance, and its distributed pushback mechanism can significantly reduce the occupation of network resources by DDoS attacks.

Our specific contributions include:

- We introduced an innovative DDoS attack detection and mitigation framework that can be efficiently deployed on programmable switches. By optimizing resources and utilizing intelligent detection, it achieves the ability to quickly identify and respond to complex DDoS attacks in high-traffic environments.
- We designed an innovative Decision Tree - Constrained Threshold Segmentation (DT-CTS) pruning algorithm that maintains high model accuracy while reducing thresholds by 70%. Unlike traditional pre-pruning or post-pruning methods, DT-CTS is specifically tailored to P4 switch hardware characteristics by constraining decision thresholds during the tree-building phase to align with pipeline and match-action table structures. This optimization significantly reduces model complexity, making it easier to deploy in resource-constrained PDP, while also enhancing detection efficiency.
- We use carefully selected features for training using DT-CTS, and only three features are needed to achieve 98% F1 score, while optimizing hardware resource utilization and greatly reducing the occupation of flow table entries of programmable switches.

[1]SISTAR combines 'switch' and 'star', symbolizing how the switches' protection in the network shines brightly, like stars in the sky.

- We proposed a protection measure for switch clusters, utilizing alert information and a pushback defense mechanism to enable distributed model deployment across multiple switches. This approach aims to push DDoS traffic detection and mitigation to the network edge, minimizing the impact on network resources, and ensuring network availability for legitimate traffic.

2 Background

2.1 DDoS Attacks Solutions

DDoS attacks remain a critical challenge in network security, leveraging botnets to unleash massive traffic loads rapidly [1, 45]. Recent trends indicate not only an exponential increase in attack volume but also a diversification in strategies, particularly targeting application layers like HTTP and DNS [15, 38]. These attacks aim to deplete server resources, preventing legitimate access.

To counteract evolving threats, numerous solutions have been proposed. Traditional defense, such as firewalls [11, 57] and intrusion detection systems (IDS) [12, 61], rely on static rules to filter malicious traffic. While effective against common attacks, these rules lack the adaptability to handle evolving threats. Additionally, updating and maintaining static rules require substantial human effort, making rapid response to complex attacks challenging.

Machine learning (ML) and Deep Learning (DL) approaches have gained traction for DDoS detection, yet they often require considerable time to identify threats due to performance and real-time processing constraints. For instance, frameworks like DeepDefense [69] employ convolutional and recurrent neural networks to shift detection from packet-level to window-level, improving accuracy. Other methods [40, 48] integrate SVMs and autoencoders for traffic classification. Despite their potential, these centralized approaches can introduce latency and higher processing costs, hindering rapid response to large-scale attacks in enterprise networks.

Pushback [21] is a defensive technique for mitigating DDoS. The core idea of pushback is to push traffic control and filtering functions from the attacked target server to the upstream nodes of the network. Pushback reduces the burden on the target server by filtering or throttling malicious traffic at upstream network nodes, closer to the attack source. Specifically, the pushback mechanism blocks or limits the propagation of malicious traffic by detecting and identifying abnormal traffic patterns and installing filtering rules on upstream routers or switches [7, 34, 49]. This method can not only reduce the possibility of attack traffic reaching the target server in the early stage, but also spread the burden of traffic filtering and improve the defense ability of the whole network. Despite its effectiveness, current implementations on PDP are limited.

2.2 DT on PDP

Traditional network devices rely on fixed hardware architecture (ASIC) to perform tasks such as packet forwarding, and their functions and performance are limited by the factory design, which limits their flexibility and adaptability, and it is difficult to cope with dynamic network requirements. Software Defined Networking (SDN) [68] achieves centralized management and configuration flexibility by separating the control plane and the data plane. However, the fixed function of the traditional data plane is still the bottleneck. Therefore, PDP technology comes into being.

Table 1: ML Frameworks Deployed on PDP

Framework	Model	Application
NetBeacon [77]	RF, XGB	In-Network Classification
Planter [76]	RF, XGB	In-Network Classification
SwitchTree [27]	RF	In-Network Classification
IIsy [75]	DT, RF, XGB	In-Network Classification
Mousika [64]	DT	In-Network Classification
Soter [63]	DT	Attack Detection
pHeavy [73]	DT	Heavy Flows Prediction
Netpixel [56]	DT	Image Classification
SISTAR	DT, RF	DDoS Attack Detection

PDPs based on the Protocol-Independent Switch Architecture (PISA) offer unprecedented flexibility in traffic control [30]. As shown in Figure 1, PDP redefine packet processing through a pipeline consisting of parsing, match-action, and forwarding stages. In the parsing stage, packet headers are extracted and converted into an actionable internal representation, identifying fields such as Ethernet, IP, and TCP/UDP. The match-action stage is the core of packet processing, utilizing units like TCAM or SRAM for efficient rule matching, while arithmetic logic units (ALUs) perform predefined operations such as header modifications or statistics updates. Finally, packets are deparsed and transmitted. As a result, this architecture enables complex traffic analysis and supports advanced network functions, while achieving higher throughput than traditional software-based packet processing.

Although PDP offer a high degree of flexibility, they still face significant limitations. For example, in the case of Tofino1 [60], the architecture supports only a fixed 12 processing stages, while complex operations may need to span multiple stages. In addition, its data processing capability is limited in the bit width, and it does not support data processing over 32 bit width. Resource constraints are also a critical issue, with a single SRAM pipeline being only 120MB and a TCAM being less than 6.2MB. These limitations affect the scalability and adaptability of the PDP architecture, and require effective resource management and optimization in the design and implementation.

Due to their complexity, neural networks are challenging to implement on switches with limited hardware resources [9]. This is because switches have limited computational resources and lack support for complex floating-point operations and variable bit-width data processing. A decision tree (DT) is a machine learning model for decision-making based on input features. In a DT model, each node represents a decision rule that routes the data into different branches based on specific input characteristics. Such a model is well suited to the programmable switch paradigm because its hierarchical decision structure highly fits the match-action pipeline of the switch. In switches, packet processing is performed through a sequence of operation stages, each of which can correspond to a node in the DT. In this way, the condition check of the DT can be mapped to the processing stage of the switch to decide the next action according to the processing of the packet header information. Therefore, many traffic classification frameworks have adopted the DT model, as shown in Table 1.

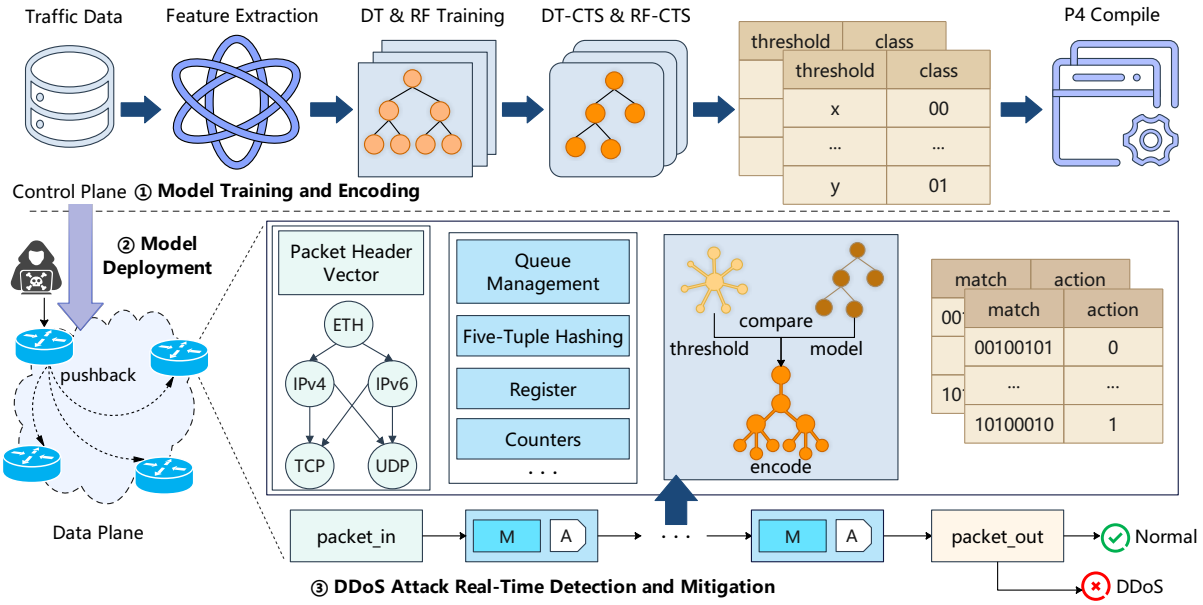


Figure 2: Overview of SISTAR.

In IIsy [74], both supervised and unsupervised learning were introduced into the match-action pipeline, using lookup tables to store classification results—marking the first successful mapping of ML to programmable switches. SwitchTree [27] mapped random forests (RFs) onto programmable switches, estimating complex features for network attack detection via hash mapping. Planter [76] proposed an algorithm for efficient mapping of ensemble models to PDP, increasing the number of trees that can be mapped to a network device by overlapping trees within match-action tables.

Despite significant efforts to deploy DTs on PDP for high accuracy in DDoS attack detection, hardware resource constraints remain a challenge. Therefore, this paper explores a more efficient method aimed at achieving high throughput and satisfactory detection accuracy by reducing the required learning features and switch resources.

3 Approach Overview

3.1 Motivation

Currently, frameworks for DDoS attack detection using PDP can be categorized into two types: general-purpose in-network classification frameworks and frameworks focused on DDoS attack detection. For the generic classification framework, Mousika [64] transfers knowledge from other models into binary decision trees by redesigning the algorithm, but only considers packet-level features, limiting in-depth analysis of DDoS attacks. Although NetBeacon [77] improves model accuracy and deployment efficiency, it requires flow length estimation to distinguish long and short flows, which may impact state storage efficiency. The IIsy [75] supports the deployment of hybrid models, but the frequent switch and server interactions increase latency and complexity, and its encoding method has certain limitations. Soter [63] employs a lightweight deep neural network, called Branch Convolution Net (BCN), that runs on the switch CPU, resulting in significant resource consumption and increased latency.

In frameworks focused on DDoS attack detection, existing models typically do not utilize machine learning deployment. When confronting new or unknown attacks, some traditional solutions [72] may struggle to update and deploy new strategies promptly, leading to defense vulnerabilities. Additionally, some approaches [31] may require frequent updates and reprogramming of switches, increasing the management and configuration burden for large-scale deployments. Therefore, there is a critical need for a method that enables rapid deployment and updating of defense strategies, suitable for large-scale deployment.

3.2 Challenge

3.2.1 Challenge 1 (C1). **The challenge of balancing detection accuracy and resource consumption.** The primary challenge existing models face in DDoS detection is the trade-off between detection accuracy and resource consumption. On the one hand, in order to improve the detection accuracy, a large number of features need to be extracted to train the machine learning model. This not only increases the complexity and scale of the model but also leads to higher computational costs. Therefore, it is crucial to find the optimal balance between the number of features and the accuracy of the model. On the other hand, some models consume substantial resources during actual deployment, especially in programmable switches, where a large number of entries can occupy excessive space and affect its operation efficiency. Therefore, optimizing models to reduce resource usage remains a key challenge.

3.2.2 Challenge 2 (C2). **The challenge of insufficient cooperative processing mechanisms between switches.** In the face of large-scale DDoS attacks, a single device often struggles to independently mitigate the attack traffic across the entire network, and multiple switches need to work together efficiently. However, the

Table 2: Hardware Resource Demand Categories

Resource	Feature Details
Low	Destination Port, Total Forward Packets, Forward Header Length, Init Window Bytes Forward/Backward, Min Segment Size Forward,
Medium	Total Length, Max, Min, Mean, Std, Subflow Forward/Backward Packets, Bytes,
High	Flow Duration, Flow Bytes/s, Flow Packets/s, Flow IAT Mean, Std, Max, Min, Forward IAT Mean, Std, Max, Min, Backward IAT Std, Max, Active Mean, Std; Idle Mean, Std, Max, Min, Forward Packets/s, Backward Packets/s,

existing system lacks effective multi-switch cooperative processing mechanism. Especially after a DDoS attack is detected, there are still deficiencies in how to quickly and effectively transfer information between multiple network nodes and coordinate actions to mitigate the impact of the attack.

3.3 System Overview

SISTAR is a distributed framework designed for detecting and mitigating DDoS attacks, as illustrated in Figure 2. It operates through three key phases to achieve efficient attack detection and mitigation.

Training Phase. During the training phase, to address **C1** 3.2.1, SISTAR utilizes a carefully selected feature set and the DT-CTS (Decision Tree - Constrained Threshold Segmentation) model. This phase optimizes feature selection to maintain high detection accuracy while significantly reducing model complexity and resource requirements. It ensures the model can accurately identify attack behaviors in dynamic network environments. When a new attack is encountered, the model can be quickly retrained and deployed to ensure the detection ability of DDoS attacks.

Deployment phase. The trained models are distributed across multiple switches. Each switch deploys a tailored version of the model based on its role and network location to optimize resource utilization. By adjusting model thresholds, the system ensures efficient operation on programmable switches, thereby improving the overall network's response speed and processing capability against DDoS attacks.

Detection Phase. During the detection phase, the system performs real-time packet analysis. Switches extract static and flow-level features from packets to identify potential DDoS attacks. To address **C2** 3.2.2, if suspicious activity is detected, the system generates alert packets and distributes them to other switches for further analysis. Once the attack is confirmed, SISTAR executes a pushback mechanism to push the traffic control to the upstream nodes of the network, effectively reducing the burden on the target server and mitigating the DDoS attack.

Through the collaborative work of these three phases, the SISTAR framework provides efficient, updatable, and low-footprint DDoS attack detection and mitigation capabilities in complex network environments. To our knowledge, SISTAR is the first framework to integrate machine learning-based detection with a distributed pushback mechanism on the programmable data plane for comprehensive DDoS protection.

Table 3: Features in SISTAR

Feature Category	Definitions
Packet Level	port-related, Flag, ttl, header and window size, packet size, etc.
Flow Level	time intervals, packet/flow Rates, flow Statistics, etc.

4 Feature Engineering

4.1 Feature Extraction

In the data plane, feature extraction is crucial to ensure compliance with hardware and P4 language constraints [42]. While traditional machine learning focuses on improving model accuracy, feature selection in the data plane must also consider hardware resource limitations. Therefore, our goal is to extract fine-grained features while meeting these constraints.

Network traffic features are divided into packet features and flow features. Packet features are extracted from the header of individual packets, with specific selections depending on the application scenario. Flow features provide richer information. Studies [77] have shown that flow features significantly enhance detection accuracy, but maintaining flow state requires additional hardware resources, such as registers. Thus, balancing resource utilization with detection effectiveness is critical.

Table 2 lists the hardware resources required to map features onto the data plane. Furthermore, we calculated feature importance and ultimately selected the relevant features as shown in Table 3.

Specifically, given a fitted random forest prediction model m and a dataset D, we can calculate the reference score s. The importance i_j of feature f_j in model m is calculated using the following formula:

$$i_j = s - \frac{1}{K}\sum_{k=1}^{K} s_{k,j} \tag{1}$$

For each k in 1, …, K: randomly shuffle the values in the j-th column of D. The shuffled data set is indicated as $\overset{*}{D}_{k,j}$, and $s_{k,j}$ is the model reference score on $\overset{*}{D}_{k,j}$.

To construct a high-accuracy model, we selected features from both individual packets and flow-level statistics. While flow features consume more hardware resources, the overhead remains within acceptable limits. This approach optimizes feature selection within resource constraints, ensuring model efficiency and accuracy.

We also explored the impact of different numbers of features, decision tree depths, and tree counts on model accuracy to optimize feature and tree hyperparameters. As shown in Figure 3, in the first experiment, 9 trees with depth of 3/4/5/6/7. In the second experiment, tree depth at 7 with trees of 1/3/5/7/9. Both experiments used feature counts ranging from 3 to 7. Results show that when the number of model features is small, tree depth has a greater impact on model accuracy than the number of trees.

4.2 DT-CTS Training

The effectiveness of decision trees in classification has been widely verified. Although the existing algorithms can directly control the depth of the tree through the parameters, what we want to do is to

Algorithm 1: DT-CTS Training

Require: Feature matrix X, target vector y, maximum thresholds per feature T, maximum depth D
Ensure: Optimal threshold and impurity for the selected feature

```
Initialize thresh_cnt as an empty dictionary
function BestSplit(X, y, feat)
if feat not in thresh_cnt then
    thresh_cnt[feat] ← 0
end if
if thresh_cnt[feat] ≥ T then
    return None
end if
Initialize best_thresh ← None,
best_impurity ← ∞
for each thresh in unique values of X[:, feat] do
    impurity ← impurity(X, y, feat, thresh)
    if impurity < best_impurity then
        best_impurity ← impurity
        best_thresh ← thresh
    end if
end for
if best_thresh ≠ None then
    Inc thresh_cnt[feat]
end if
return best_thresh, best_impurity
```

make it further fit the matching and action operation of the switch in the classification processing, and reduce the judgment of the switch for the feature value as much as possible. To this end, we design Algorithm 1 called Decision Tree - Constrained Threshold Segmentation (DT-CTS).

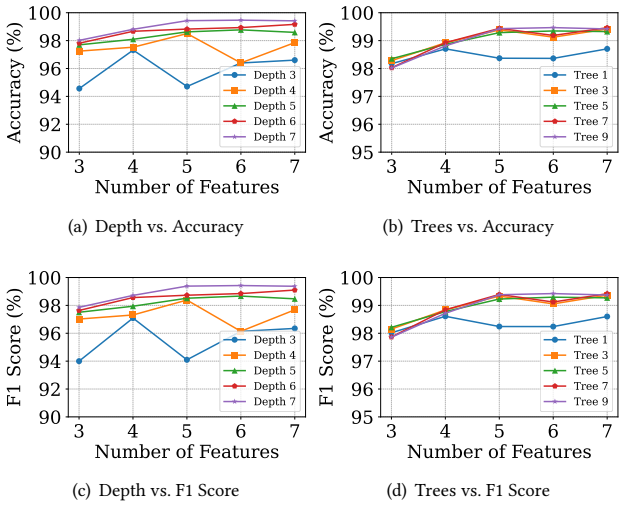
The training dataset consists of a feature matrix X and a target vector y, where $X \in \mathbb{R}^{m \times k}$, m is the number of samples, and k is the number of features. Our goal is to generate an optimized decision tree that partitions the dataset into leaves with unique class labels.

First, we initialize a dictionary *threshold_count* to record the number of split thresholds used for each feature. For each feature f, we invoke the function BestSplit to identify the optimal split threshold t^*. In BestSplit, we iterate over all unique values t of feature f and calculate the Gini impurity $G(t)$:

$$G(t) = 1 - \sum_{i=1}^{C} p_i^2 \tag{2}$$

where C is the number of classes, and p_i is the probability of class i. We select the threshold that minimizes $G(t)$ as the optimal split point. If a feature's threshold count reaches the maximum limit T, no further splits are made. Once the optimal threshold t^* is determined, we update *threshold_count* to ensure it does not exceed the limit. A schematic diagram of its model generation is shown in Figure 4.

Benefits. The advantage of using the DT-CTS model, along with the constraint threshold segmentation for the RF-CTS model generated by DT-CTS, is the reduction of redundant decision nodes.



(a) Depth vs. Accuracy (b) Trees vs. Accuracy

(c) Depth vs. F1 Score (d) Trees vs. F1 Score

Figure 3: Impact of tree depth and number on detection performance. (a) and (c) show the effect of varying tree depth; (b) and (d) show the effect of varying the number of trees.

By limiting the number of split thresholds, the structure of the decision tree becomes more streamlined. When DT-CTS or RF-CTS is mapped onto a switch, this significantly reduces the computational and storage demands on the switch. Fewer decision nodes mean that the switch needs to compare fewer thresholds and store fewer rules, thereby conserving hardware resources and enhancing processing speed.

Especially when encoding DT using an efficient method like IIsy [75], the number of encoding bits is fixed. However, if a decision tree has too many thresholds for a particular feature, it may lead to insufficient bit length. This issue is particularly pronounced when there are few features but the tree is deep. Simply increasing the bit length would significantly increase table entry consumption. Therefore, if the number of thresholds for a single feature is too high, the required bit length also increases, making it challenging to meet encoding requirements without limiting the number of thresholds in the generated tree.

4.3 Feature Encode

After reducing the redundancy of DT nodes, SISTAR uses state-of-the-art encoding methods [75] to map the model to programmable switches. This encoding method is independent of the tree's depth and only related to the features used. By distinguishing between different thresholds of the same feature within a tree, we achieve correct encoding without requiring multi-stage processing; all matches can be completed in a single stage. Specifically, we encode the path from the root to the leaf nodes, with the number of leaf nodes determining the number of encodings needed.

Taking a single DT-CTS as an example, as shown in Figure 5, data is segmented into different intervals by comparing specific feature thresholds, and each interval is encoded. Specifically, for a feature x, suppose the feature thresholds are t_1 and t_2. The encoding rules are as follows: if $x < t_1$, the encoding is 00; if $t_1 < x < t_2$, the encoding is 01; if $t_2 < x < t_3$, the encoding is 10; and if $x > t_3$, the encoding

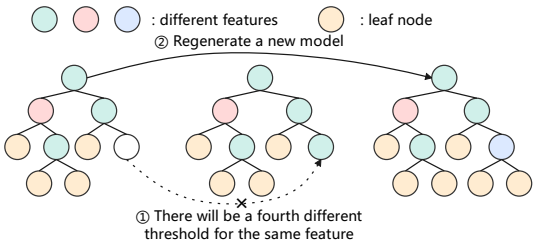
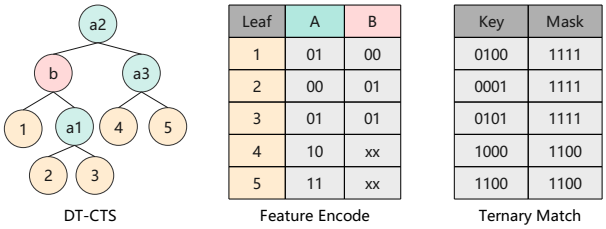


Figure 4: An example of DT-CTS model generation.



Leaf	A	B
1	01	00
2	00	01
3	01	01
4	10	xx
5	11	xx

Key	Mask
0100	1111
0001	1111
0101	1111
1000	1100
1100	1100

Figure 5: An example of DT-CTS model encoding and matching table, where a1 < a2 < a3.

is 11. In the DT-CTS, each node splits based on specific thresholds (such as $a1, a2, a3, b$), ultimately generating feature encodings at the leaf nodes. For leaf 2, the value of feature A is less than threshold $a1$, so feature A is encoded as 00. The value of feature B is greater than threshold b, thus feature B is encoded as 01.

Assuming the set of leaf nodes in the DT-CTS is L, each leaf node $l \in L$ corresponds to a set of feature encodings E_l, where $E_l = (e_a, \ldots, e_f)$ represents the encoding values for each feature. Using ternary matching can further optimize matching efficiency compared to other methods, leading to superior performance in single-stage processing. Finally, the feature encoding is mapped into a table, where the key represents the feature encoding and mask specifies important bits. The '0' in the mask represents the "don't care" bit, which can match any value.

5 Distributed Deployment

Traditional model deployment is typically concentrated on a single device or a few nodes, which can lead to resource bottlenecks and limited protection effectiveness against large-scale distributed attacks. To address this, we propose a distributed model deployment strategy aimed at optimizing resource utilization and system efficiency. By deploying heterogeneous machine learning models, such as DT-CTS and RF-CTS, across different programmable switches, we achieve comprehensive protection against DDoS attacks. This distributed deployment not only enhances the flexibility of detection and response but also maximizes the potential of each node.

Our solution is applicable to data centers and enterprise networks within a single administrative domain, relying on the collaborative operation of multiple programmable switches. Based on network location and resource capacity, each switch deploys different models: comprehensive models are deployed on switches near the core for detailed traffic inspection and complex attack analysis; lightweight models are deployed on switches near the gateway for rapid identification and filtering of abnormal traffic, reducing latency and resource consumption. This strategy not only optimizes overall detection performance but also alleviates the load on individual switches.

Listing 1: P4 code snippet of the model forwarding function

```
apply {
    if (!hdr.alert.isValid()) {
        // Analyze and extract features
        parser_feature.apply();
    }
    // DDoS detection phase
    DDoS_ternary.apply();
    if (meta.ddos_detected == 1) {
        // model encoding for forwarding
        meta.forward = meta.bin_feature;
    } else {
         send_packet.apply();
    }
}
```

In conjunction with this distributed model deployment strategy, we further optimize the pushback mechanism. When a DDoS attack is detected, the system can intelligently trace back to the source of the attack traffic and dynamically adjust the defense strategies of each node. This mechanism can not only quickly respond to attacks, but also effectively reduce the burden of the overall network and ensure the stability and reliability of the network.

5.1 Alert Generation

In our architecture, the alert generation mechanism is designed to efficiently identify and mitigate DDoS attacks without relying on the control plane. By implementing multi-level detection in the PDP, switches can autonomously detect attacks and initiate pushback mitigation. This process is achieved by sending specific alert packets, which switches use to notify neighboring switches and trigger a coordinated defense mechanism. The alert packets utilize a custom header format for efficient and accurate communication between switches. This format supports the transmission of basic alert information and includes the alert severity level of the DDoS attack. After the packet arrives at the switch, it is processed roughly through the following steps.

Packet Classification. Upon receiving a packet, the switch first classifies it to determine if it is a regular packet. For regular packets, the switch extracts features for further DDoS attack detection.

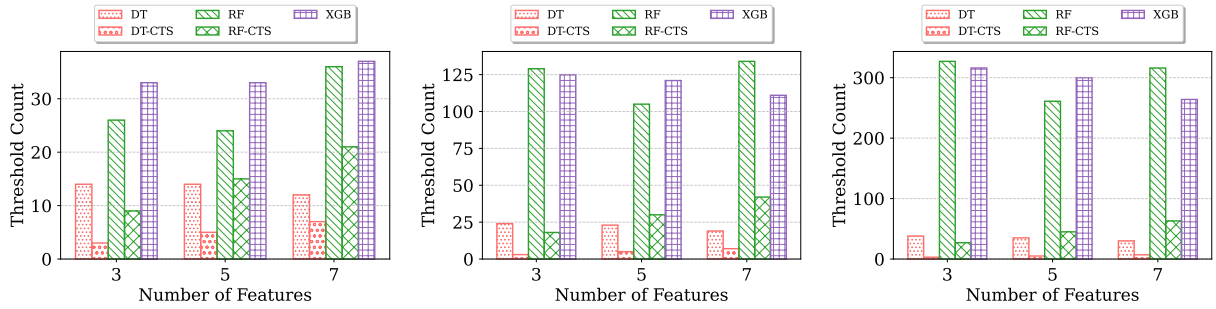
DDoS Attack Detection. Using a machine learning model, the switch analyzes the extracted features to determine if the packet is part of a DDoS attack. If deemed normal, the packet is forwarded as usual; if suspicious, an alert packet is generated.

Alert Packet Processing. When a switch receives an alert packet, it determines the alert level using its features. The alert packet uses a custom header format to ensure accurate and efficient information exchange between switches.

Pushback Mechanism. Upon detecting a DDoS attack, the switch sends specific alert packets to upstream switches, triggering the pushback mechanism to mitigate the attack at its source.

5.2 Model Pushback

When the alert level of an alert packet reaches a set threshold, the pushback mechanism is activated. The switch then sends alert packets upstream, contains flow table information relevant to the DDoS attack. Upon receiving this alert packet, the upstream switch installs the flow table information into its own entries, enabling early detection of the DDoS attack.



(a) Small Model (b) Medium model (c) Large Model

Figure 6: Characteristic threshold counts required at different model scales.

Listing 2: P4 code snippets for encoding and matching

```
// Partial feature encoding
action parser_feature() {
    meta.bin_feature[1:0] = meta.
         Destination_Port_class;
    meta.bin_feature[3:2] = meta.
         Init_Win_bytes_forward_class;
    meta.bin_feature[5:4] = meta.
         Fwd_Header_Length_class;
    meta.bin_feature[7:6] = meta.
         Packet_Length_Mean_class;
    meta.bin_feature[9:8] = meta.
         Flow_Packets_persecond_class;
}
// Matching based on features
table DDoS_ternary {
    key = {
        meta.bin_feature: ternary;
    }
    actions = {
        malicious_packet_forward;
        benign_packet_forward;
        NoAction;
    }
    size = 1024;
    default_action = NoAction();
}
```

Listing 1 shows the code for the forwarding model. First, check if the packet is an alert packet. If not, process it as a regular packet, extract features, and perform DDoS detection. If it is an alert packet, forward the matched feature encoding based on the detected DDoS attack. When the alert count reaches a preset threshold, the pushback mechanism is triggered. If no attack is detected, the alert count decreases to zero, and the switch generates and forwards a normal packet.

5.3 Prototype Implementation

We implemented a prototype[2] for DDoS attack detection on the programmable switch Tofino1 [60] and conducted a simulation of SISTAR based on BMv2 [43]. Tofino1 has four pipelines, each divided into 12 MAU stages, with each stage featuring 10MB of SRAM and 0.528MB of TCAM. To effectively evaluate the performance of the machine learning model and the efficiency of the pushback mechanism, we developed multiple data plane files using $P4_{16}$ [42] and wrote several control plane scripts in Python. We did not perform actions such as deactivating logging to improve the performance of BMv2.

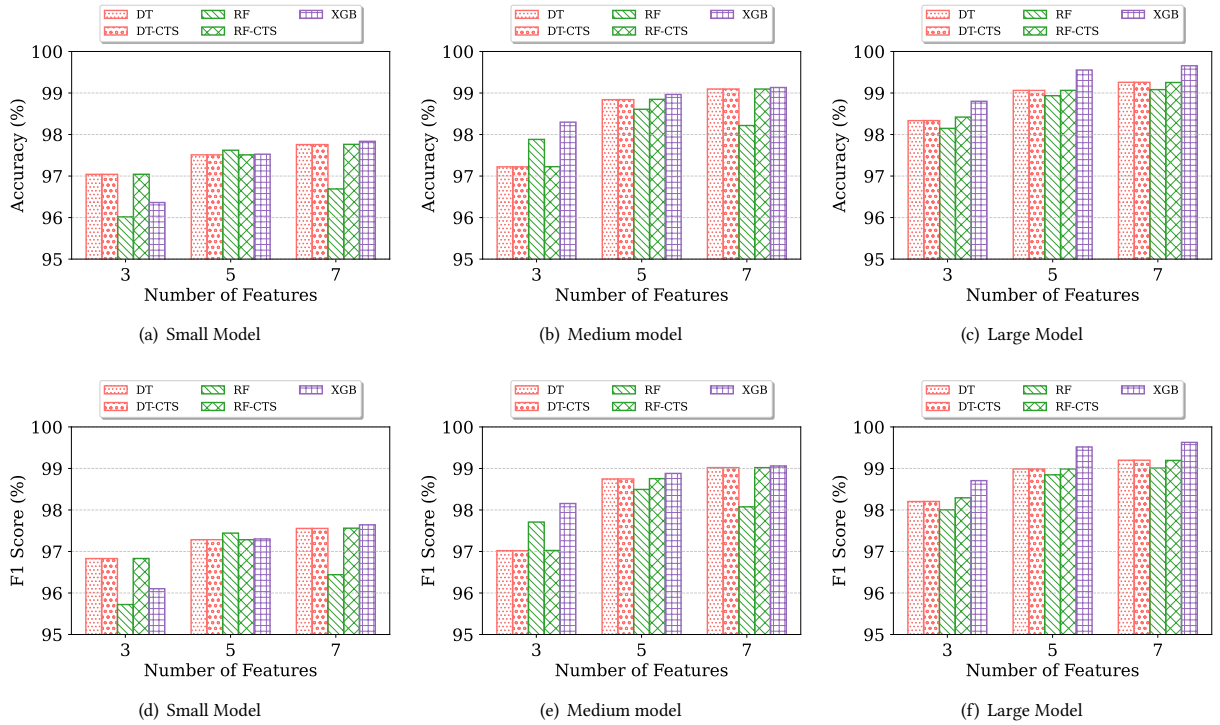
[2]https://github.com/hugo0819/SISTAR.

Listing 2 demonstrates the model deployed on a single switch. The features used by the machine learning model are encoded into *class* based on their thresholds, which are then converted into ternary matching actions for the $bin_feature$. Incoming packets to the switch are parsed to extract their features, which are compared against the model's thresholds to obtain their classification code. If the code matches the $bin_feature$, the corresponding action, such as $malicious_packet_forward$ or $benign_packet_forward$, is executed to forward the packet through its designated port. This implementation ensures efficient resource usage and rapid response, critical for real-time DDoS mitigation.

6 Evaluation

To demonstrate the feasibility and performance of SISTAR, we conducted the following experiments:

- We evaluated the performance of the improved machine learning algorithms, focusing on comparing the original DT and RF with the modified DT-CTS and RF-CTS. The results showed that reducing threshold usage maintained accuracy and led to more stable performance.
- We evaluated the models on six datasets, the results indicate that DT-CTS and RF-CTS exhibit good generalization.
- We conducted a comprehensive evaluation of SISTAR's DDoS detection accuracy, demonstrating that it outperforms existing state-of-the-art models.
- We explored SISTAR's usage of hardware resources such as SRAM, TCAM, and flow table entries. The results indicated that memory resource usage was less than 3%, with minimal flow table entry consumption.
- In a switch cluster environment, we evaluated the defense performance of SISTAR and its backtracking mechanism against real and synthetic attacks. The results demonstrate excellent performance, accurately distinguishing between normal and abnormal traffic.
- The effectiveness of the SISTAR pushback mechanism in mitigating abnormal traffic across the network was evaluated, and simulation results show that it can reduce attack traffic bandwidth utilization by up to 40%.



(a) Small Model (b) Medium model (c) Large Model

(d) Small Model (e) Medium model (f) Large Model

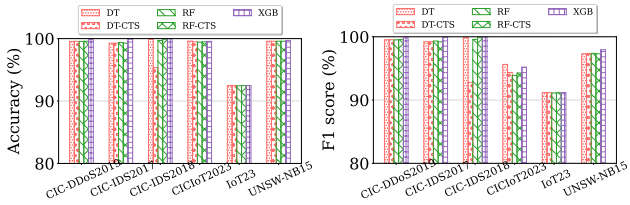
Figure 7: Comparison of model performance across different scales.

6.1 DT-CTS and RF-CTS Performance

We used the CICIDS2017 dataset [54], a standard dataset containing traffic data of multiple DDoS attack types. We selected the subset of samples on Wednesday, July 5, 2017, which contains multiple DoS attacks. The dataset was processed on a server equipped with an Intel(R) Xeon(R) Platinum CPU @2.5GHz processor, using Python3 and the scikit-learn (Sklearn) library [46]. The data was split into training and testing sets at a 4:1 ratio.

We conducted a comprehensive evaluation of DT-CTS and RF-CTS for three different model sizes: small (3 trees, depth 4), medium (6 trees, depth 5), and large (9 trees, depth 6). In the experiments, we compared five models: DT, DT-CTS, RF, RF-CTS, and XGBoost (XGB). Our evaluation is carried out in the following three aspects: 1) Number of thresholds: that is, the sum of the number of thresholds for each feature in the model. 2) Accuracy: that is, the percentage of correctly classified test samples over the total number of their classes. 3) F1 score: the harmonic mean of Precision and Recall.

Firstly, we explored the significant advantages of DT-CTS and RF-CTS in reducing the number of thresholds. As shown in Figure 6, by varying the number of features (3, 5, 7), we compared the threshold counts required by each model at different scales. It is evident from the figure that DT-CTS and RF-CTS demonstrate superior efficiency in feature selection compared to traditional DT and RF models. Specifically, in the small model 6(a), as the number of features increases, the threshold counts for DT-CTS and RF-CTS are significantly lower than those for DT and RF. This trend becomes even more pronounced in the medium 6(b) and large models 6(c). Particularly in the large model, the threshold count for



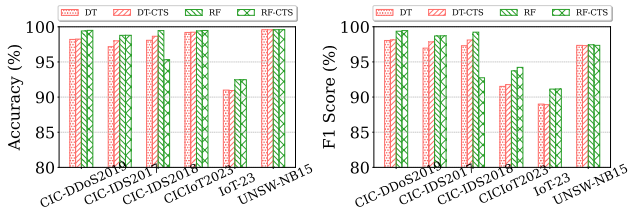
(a) Accuracy comparison (b) F1 score comparison

Figure 8: Performance of five models on the six datasets.

RF exhibits exponential growth with an increasing number of features, whereas RF-CTS effectively controls the growth of threshold counts, highlighting its effectiveness in complex models.

The XGBoost (XGB) shows relatively higher threshold counts across all model scales, indicating greater complexity in feature selection. In contrast, DT-CTS and RF-CTS successfully reduce model complexity through their feature selection mechanisms while maintaining good performance. This result further confirms the significant advantages of DT-CTS and RF-CTS in feature selection and model simplification, especially when handling high-dimensional feature data, where their efficiency is particularly noteworthy.

Secondly, we compared the performance of DT-CTS and RF-CTS with traditional models in terms of accuracy and F1 score. Figure 7 illustrates the performance of each model across different scales and feature numbers. In the small model (Figures 7a and 7d), DT-CTS shows accuracy and F1 scores nearly identical to those of DT, indicating that the algorithm maintains stable classification performance while significantly reducing the number of thresholds.



(a) Accuracy comparison (b) F1 score comparison

Figure 9: 5-fold cross-validation on the six datasets.

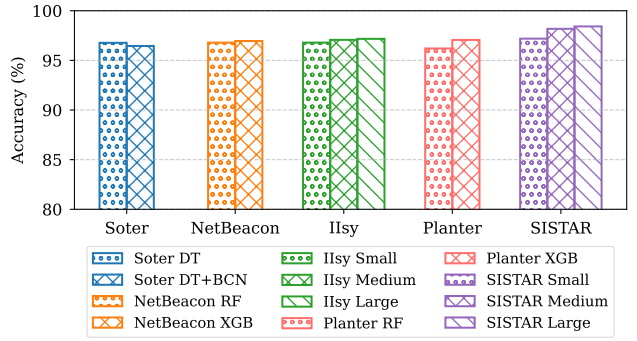


Figure 10: Comparison of DDoS attack detection performance across different frameworks on programmable switches.

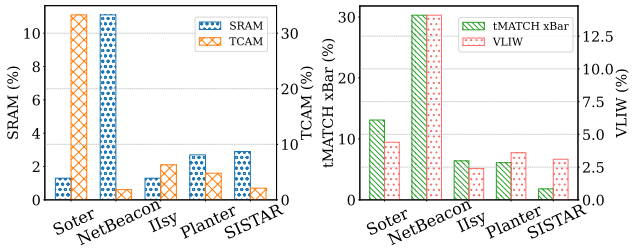
However, RF-CTS exhibits considerable differences from RF in small-scale models, especially with fewer features, possibly due to RF's excessive threshold count leading to classification instability. As the model scale and feature number increase (Figure 7(b), 7(c), 7(e), 7(f)), RF-CTS shows more stable performance improvements, highlighting its potential advantages in complex models.

In medium and large models, RF-CTS gradually achieves performance comparable to RF, and in some cases even surpasses it, particularly in terms of F1 score with more features. This suggests that RF-CTS can better balance precision and recall when handling complex feature sets.

Additionally, XGBoost consistently maintains high accuracy and F1 scores across all model scales, demonstrates strong performance in handling multi-feature data. However, DT-CTS and RF-CTS, through effective feature selection mechanisms, achieve lower model complexity while maintaining competitive performance. These results further validate the adaptability and advantages of DT-CTS and RF-CTS across different model scales.

6.2 Generalization Across Datasets

To further validate the practical performance of DT-CTS and RF-CTS, we used six datasets: CIC-IDS2017 [54], CIC-IDS2018 [54], CIC-DDoS2019 [55], CICIoT2023 [39], IoT23 [53], and UNSW-NB15 [37]. CIC-IDS2017 and CIC-IDS2018 simulate DoS and DDoS attacks in enterprise network environments; CIC-DDoS2019 targets high-pressure DDoS scenarios in cloud services, including various DDoS attack variants; CICIoT2023 and IoT23 focus on botnet DDoS attacks in IoT and industrial IoT environments, featuring the latest attack variants and protocol exploitation DoS attacks; UNSW-NB15 represents traditional DoS attacks in university campus networks.



(a) SRAM & TCAM usage (b) tMATCH xBar & VLIW usage

Figure 11: The usage of memory resources and computing resources on hardware switches by different frameworks.

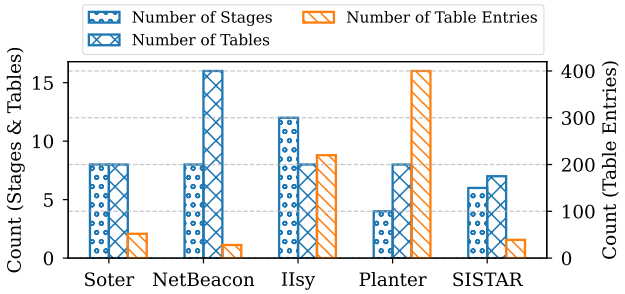


Figure 12: Comparison of stage, table and table entries.

We first evaluated the performance of DT, DT-CTS, RF, RF-CTS, and XGB on six datasets, as shown in Figure 8. Except for IoT23, all models achieved 95% accuracy on the other datasets. Notably, compared to models without the CTS algorithm, the CTS algorithm reduces model complexity while maintaining accuracy and F1 score declines of no more than 1% on most datasets.

To further validate the generalization capability of the models and assess their stability and consistency across different data splits, we conducted 5-fold cross-validation on DT, DT-CTS, RF, and RF-CTS models across the six datasets. The results showed in Figure 9, except for IoT23, the DT-CTS and RF-CTS models achieved 95% accuracy on most datasets, demonstrating excellent generalization ability. Overall, DT-CTS and RF-CTS maintained performance comparable to the original models (DT and RF) while significantly reducing the number of thresholds.

On the IoT23 dataset, all models performed poorly (about 91%). This is primarily due to the imbalance in the dataset, where the ratio of normal (benign) to abnormal (malicious) traffic is 1:2. Additionally, the total number of data flows is small, resulting in insufficient benign data for effective learning.

6.3 DDoS Detection Accuracy

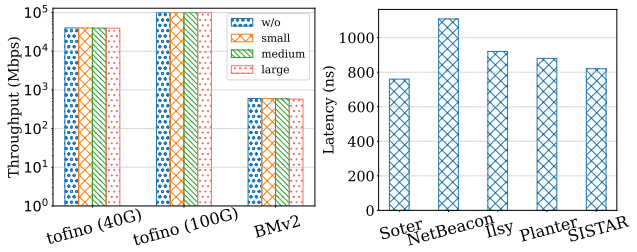
We simulated traffic transmissions using network topologies on Linux servers. To ensure fairness, we deployed the SISTAR framework without the backtracking mechanism on a programmable switch, Tofino1, connected to two hosts. One host replays the Wednesday pcap data from CICIDS2017 using tcpreplay, while the other host captures the packets received from the switch.

We compared our approach with Soter [63], NetBeacon [77], IIsy [75], and Planter [76]. The register size is uniformly set to 8192,

Table 4: Resource Utilization Distribution Across Pipeline Stages

Resource Type	Total	Stage 0	Stage 1	Stage 2	Stage 3	Stage 4	Stage 5
SRAM	2.9%	5.0%	20.0%	3.8%	5.0%	0	1.3%
TCAM	2.1%	8.3%	0	0	0	0	16.7%
tMatch xBar	1.8%	12.1%	0	0	0	0	9.1%
eMatch xBar	2.7%	10.2%	13.3%	6.3%	3.1%	0	0
VLIW	3.1%	9.4%	9.4%	3.1%	3.1%	3.1%	9.4%
Map RAM	2.3%	4.2%	16.7%	6.3%	0	0	0
Meter ALU	10.4%	25.0%	75.0%	25.0%	0	0	0
Action Data Bus	1.5%	5.5%	4.7%	3.1%	3.1%	0	1.6%
Gateway	3.1%	12.5%	18.8%	6.3%	0	0	0

Note: Stages 6-11 are all 0.



(a) Throughput (b) Latency

Figure 13: Comparison throughput and latency.

used to store the five-tuple of flows and other feature values. For IIsy and SISTAR, three models were used: small, medium, and large, with 3, 6, and 9 trees, and depths of 4, 5, and 6, respectively. For Soter, Planter, and NetBeacon, models were trained using 9 trees with a depth of 6. Compared to packet-level detection, we focus primarily on flow-level DDoS detection.

As shown in Figure 10, SISTAR achieves the best performance of other models even with small-scale models. When deploying medium or large-scale models on the switch, its detection rate improves by 0.5%-2.3% over other models. SISTAR's accuracy surpasses previous work, attributed to our focus on DDoS attack detection and the carefully selected features for optimal detection performance. It is noteworthy that regardless of the model used, the detection accuracy on the switch is generally lower than during training. This is due to hardware limitations of the switch, which affect the precision of feature computation.

6.4 Hardware Resources

To further explore the performance of SISTAR, we considered hardware resource consumption, processing flow, packet throughput, and processing delay, and compared it with Soter [63], NetBeacon [77], IIsy [75], and Planter [76] on the same switch. All models used parameters of 9 trees with a depth of 6 for detection.

We focused on the following six aspects: 1) Memory Resources: The percentage of TCAM and SRAM used. 2) Computational Resources: The percentage of tMatch xBar and VLIW used. 3) Program Resources: The number of stages, tables, and table entries. 4) Single-stage Resource: The consumption of key resources at each stage of the pipeline. 5) Throughput: The throughput performance of SISTAR on Tofino and BMv2. 6) Processing Delay: The time taken for packets to be processed by the switch.

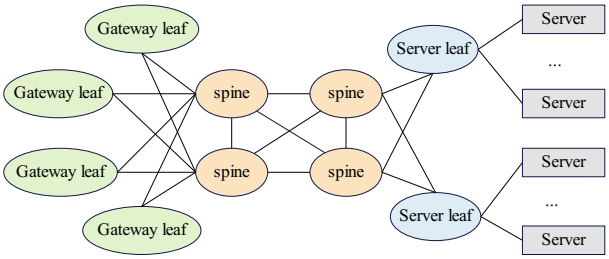


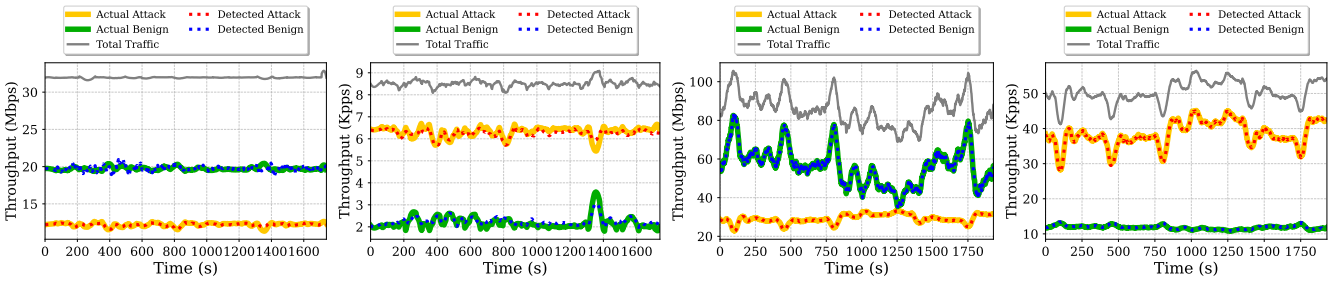
Figure 14: Test scenarios for distributed deployment and pushback performance.

Memory Resources. Memory resource consumption is shown in Figure 11(a). In programmable switches, SRAM is primarily used for storing forwarding rules, state information, and other data that require quick access. We can see that, except for NetBeacon, which consumes 11.1%, the other four frameworks consume less than 3%. Soter has the lowest consumption at 1.3%. SISTAR is 0.2% higher than Planter, which is due to the fact that SISTAR stores more state information, especially the intermediate values of feature computation. TCAM is mainly used for storing matching conditions. We observe that, except for Soter, which consumes 33.3%, the others are below 7%. NetBeacon has the lowest consumption at 1.85%, and SISTAR also performs excellently with only 2.1% consumption. This efficiency is due to our encoding method, which requires only 2 bits to encode a feature.

Computational Resource. Computational resource consumption is shown in Figure 11(b). In programmable switches, TMatch xBar represents ternary match crossbars used for ternary or range matching. VLIW stands for Very Long Instruction Word, used for executing actions. Compared to the other four frameworks, NetBeacon has the highest computational resource consumption due to its overly complex feature processing, requiring extensive computation for a single feature. SISTAR has the lowest resource consumption for TMatch xBar, requiring only 1.8%. For VLIW usage, SISTAR is at an average level, except for NetBeacon, which consumes more.

Program Resource. Program resource consumption is illustrated in Figure 12, covering three key resources: stages, tables, and table entries. Tofino1 supports up to 12 pipeline stages. Except for IIsy, which utilizes all 12, other frameworks use approximately 8. Planter is the most efficient, requiring only 4 stages, followed by SISTAR with 6. For flow table count, SISTAR again performs best, using only 7 tables, while NetBeacon uses the most at 15. This difference is closely tied to the number of features employed—SISTAR achieves higher accuracy with fewer features than other frameworks. In terms of table entries, to reach 95% accuracy, Planter requires the most (around 400), followed by IIsy with over 200. The remaining frameworks use significantly fewer entries (30–50). Notably, SISTAR maintains low table entry usage while effectively balancing stage and table consumption.

Single-stage Resource. Furthermore, we have listed the single-stage resource consumption of SISTAR, as shown in Table 4. The results indicate that SISTAR maintained extremely low occupancy rates for all key resource types, and the data for each stage show that the resource allocation was balanced. Our balanced, low-footprint design avoids creating resource bottlenecks in any per-stage, preserving capacity for other critical network functions.



(a) Real DDoS attack (bps) (b) Real DDoS attack (pps) (c) simulated DDoS attack (bps) (d) simulated DDoS attack (pps)

Figure 15: After real-world and simulated DDoS traffic enter the network topology with the distributed model deployed, the model distinguishes between benign and malicious traffic. The closer the detected curves align with the actual traffic curves, the more accurate the detection.

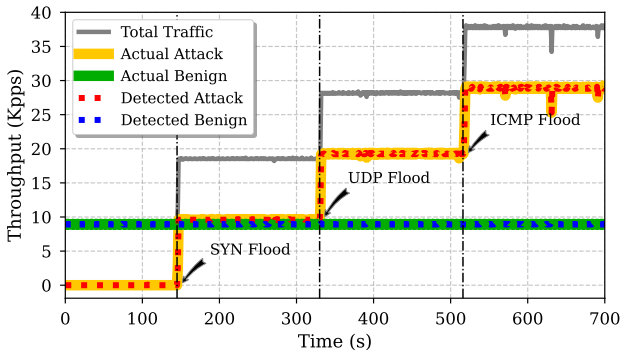


Figure 16: SISTAR's detection performance under normal traffic with sequentially injected SYN, UDP, and ICMP Flood attacks.

Throughput. Figure 13(a) presents the throughput of Tofino [60] under 40 Gbps and 100 Gbps traffic loads. Deploying models of varying scales on both Tofino and BMv2 [43] introduces no significant degradation in throughput, indicating that the SISTAR framework maintains line-rate performance across different platforms and traffic conditions. This highlights the efficiency of SISTAR's design in utilizing switch resources while preserving high detection accuracy.

Despite consistent processing capability, BMv2's throughput is relatively low. As a software-based switch, its performance is inherently constrained by CPU capabilities, multithreading efficiency, and system resource allocation. Unlike hardware switches, BMv2 lacks dedicated hardware acceleration, making it difficult to achieve high throughput, especially under heavy workloads.

Latency. Figure 13(b) presents the average packet forwarding latency across the evaluated frameworks. Soter achieves the lowest forwarding latency at 750 ns; however, it offloads suspicious packets to the control plane (CPU) for further inspection. NetBeacon exhibits the highest latency at 1108 ns, primarily due to its complex flow-level processing. While this approach enables more fine-grained feature extraction, it also introduces significant processing overhead. IIsy, Planter, and SISTAR maintain forwarding latencies in the range of 820–920 ns. Overall, SISTAR achieves low-latency packet processing without compromising detection accuracy or requiring control-plane involvement.

6.5 Real Defensive Performance

To evaluate SISTAR's defense capabilities in a distributed deployment within a cluster, we constructed a miniature Spine-Leaf topology emulation network. This topology features non-blocking forwarding and high scalability, and has been widely adopted in most modern data centers. Our simplified version (Figure 14) retains core elements, including gateway leaf switches, spine switches, server leaf switches, servers, effectively reflecting the traffic characteristics of real data centers and supporting connectivity for hundreds of servers.

We validated the system using both real-world DDoS attack traffic and simulated DDoS attack traffic, with results shown in Figure 15. DDoS attack traffic enters the network topology through any gateway leaf, traverses the spine, and targets a server connected to a server leaf. We deployed lightweight models at all gateway nodes for rapid detection and mitigation of overt attacks and complex models at the spine layer for fine-grained analysis of complex or stealthy attacks. No models were deployed on the server leaf, and the pushback mechanism was enabled.

In real-world DDoS attack traffic [54], the ratio of benign to attack packets is 2:1, with the attack packets primarily consisting of application-layer attacks such as GoldenEye, Hulk, SlowHTTP, Slowloris, and Heartbleed. In the simulated DDoS attacks, we used iperf [14] to generate normal workloads and hping3 [51] to create three types of attacks: SYN Flood, UDP Flood, and ICMP Flood, with a ratio of benign to attack packets of 1:3.

The results for real-world DDoS traffic are shown in Figures 15(a) and 15(b). Detected attack and benign traffic closely match actual conditions, with an overall detection accuracy exceeding 95%. In Figure 15(b), the ratio of benign to attack packets is approximately 2:1, while in Figure 15(a), benign traffic significantly exceeds attack traffic. Analysis shows that real-world DDoS attack packets are typically 40-100 bytes, whereas benign packets have a much larger average size, resulting in higher bps for benign traffic.

The results for simulated DDoS attack traffic are shown in Figures 15(c) 15(d). SISTAR effectively distinguishes various Flood attacks from benign traffic, with detection results almost perfectly matching actual conditions, achieving an accuracy of 99.7%. To further demonstrate SISTAR's response to Flood attacks, we tested the topology network by sequentially introducing SYN Flood, UDP

Table 5: Comparison of Deployment Effectiveness and Pushback Mechanism

s1	s2	s3	w/o pushback		w/ pushback	
			DR	ATBU	DR	ATBU
small	small	small	98.7%	13.8%	99.3%	4.1%
large	No	No	98.4%	1.6%	98.4%	1.6%
No	No	large	98.4%	47.0%	98.6%	5.9%

Flood, and ICMP Flood attacks during normal operation, as shown in Figure 16. SISTAR promptly detects the attacks and executes the pushback mechanism, with a response time of less than 1 second.

6.6 Pushback Performance

To specifically evaluate the mitigation capability of the pushback mechanism and analyze the effects of different deployment strategies, we tested its performance in Figure 14. We assumed an attack scenario where traffic enters through any gateway leaf (s1), passes through spine (s2) and server leaf (s3), and reaches the server, with 70% being attack traffic and 30% legitimate traffic. We tested three simple deployment scenarios for the model on these three types of switches: 1) small models (3 trees, depth 4) deployed on s1, s2, and s3; 2) a large model (9 trees, depth 6) deployed only on s1; and 3) a large model (9 trees, depth 6) deployed only on s3. We calculated the overall Detection Rate (DR) and the overall Attack Traffic Bandwidth Utilization (ATBU) for the experiments. DR is defined as the ratio of legitimate traffic received by the server to all traffic, while ATBU is the average proportion of attack traffic to total traffic across all link paths.

Table 5 presents our test results. Without the pushback mechanism, ATBU can reach as high as 47%, depending on the model deployment location. In contrast, with the pushback mechanism enabled, ATBU is significantly reduced by 9%–41%, while DR remains unchanged or slightly improved. With pushback enabled, switches can more quickly and proactively identify malicious traffic and reduce its bandwidth usage without compromising detection accuracy. Additionally, the results show that the closer the model is deployed to the attack source, the better the overall detection performance and the lower the ATBU.

7 Discussion

Effectiveness of SISTAR. SISTAR uses an optimized machine learning model to achieve high-accuracy DDoS attack detection. Experimental results indicate that SISTAR outperforms many existing methods in detection accuracy and effectively identifies various types of DDoS attacks. Its design focuses on resource optimization by streamlining features and utilizing efficient encoding, significantly reducing hardware resource consumption. This allows SISTAR to operate on programmable switches with minimal memory and computational resource usage.

Extensibility of SISTAR. SISTAR's detection relies primarily on model deployment, so as to respond to DDoS attacks in time. When a new type of DDoS attack is encountered, accurate detection may not be possible. Our proposed solution is to take advantage of machine learning and retrain the model. This is done by simply writing the new model encoding to the switch flow table without redeployment. In BMv2, we can directly write the threshold to the switch register, while in tofino, we still handle it in a match-action manner. This trade-off enhances extensibility. Alternatively, sacrificing scalability allows hardcoding feature thresholds into the P4 code, which will make the switch faster. However, this approach may limit the system's ability to adapt to evolving threats.

Limitations. Although SISTAR can adapt to changes through model updates, it may require time to adjust when confronted with novel attack strategies, potentially leading to a temporary decrease in detection rates. Additionally, in highly complex or dynamically changing network environments, SISTAR may encounter challenges in configuration and tuning. Different network topologies and traffic patterns may necessitate distinct model configurations to ensure optimal performance.

Security of SISTAR. SISTAR's advantage lies in its ability to support distributed model deployment, enabling users to deploy models of varying scales according to network needs, in coordination with an alert pushback mechanism. However, it has some limitations, such as only being able to push back to upstream switches. If attackers become aware of the alert pushback mechanism, they might exploit it by continuously triggering alerts. To mitigate this risk, implementing rate-limiting on alert generation can prevent excessive triggering. Fortunately, if continuously triggered, all models might eventually load onto the most upstream switch, leaving other switches unaffected. Administrators can then monitor and manage this switch using centralized logging and analysis tools to track and respond to unusual alert patterns.

Evasion Attacks. Attackers might send carefully crafted traffic to gradually infer the critical decision thresholds used by SISTAR. Continuously updating the model is a good defense. Moreover, slight random perturbations can be introduced during deployment and pushback, for instance setting the threshold T to $T \pm \delta$, ensuring that the decision boundary is no longer static or predictable.

Adversarial Threats. Switches communicate via alert packets, which may be exploited by attackers to deliberately trigger an excessive number of alerts downstream or to send forged alert packets. Faced with this situation, the custom alert-packet protocol offers baseline security since only switches with the deployed model can recognize and process the packets; all other switches treat them as unknown and drop them. We can also validate the MAC address and sequence number of alert packets to prevent attackers from creating false alarms or forging alert packets to orchestrate an "Alert Storm".

8 Related Work

In section 1, 2, we have discussed the related work most relevant to ours. In this section, we briefly introduce other related works.

DDoS Attack Detection. DDoS attacks have been extensively studied since their emergence[16, 22–24, 67]. Traditional detection methods struggle to meet the demands of detecting dynamic and complex DDoS attacks[32, 50]. ML and AI have played significant roles in DDoS detection by analyzing large volumes of network traffic data to identify anomalous patterns[3, 25, 41, 70]. These technologies can automatically adapt to new attack types, enhancing detection accuracy and efficiency.

In-Network Classification on PDP. Recent efforts have focused on deploying ML models on PDP [10, 17, 29, 59, 65]. These studies focus on enhancing the accuracy and efficiency of network traffic classification. By leveraging the high-speed data processing capabilities of programmable switches, researchers can perform complex classification tasks during packet forwarding.

AI-Driven PDP Solutions. AI has enhanced programmable data planes with advanced capabilities, enabling them to perform complex network tasks, particularly in performance optimization and resource management [18, 44], intelligent management and QoS [20, 26], as well as security and reliability [52, 66].

9 Conclusion

In this paper, we presented SISTAR, a novel framework for DDoS detection and mitigation utilizing the Programmable Data Plane. SISTAR effectively balances detection accuracy and resource efficiency by integrating the DT-CTS model. Our approach addresses the limitations of existing methods by deploying optimized machine learning models across multiple switches, enabling distributed and cooperative defense. Experimental results show that SISTAR achieves superior detection accuracy while significantly reducing hardware resource usage. The pushback mechanism further strengthens network resilience by mitigating attack traffic closer to the source.

10 Acknowledgment

This work was partially supported by the Program of Key Laboratory of Network Assessment Technology, the Chinese Academy of Sciences, Program of Beijing Key Laboratory of Network Security and Protection Technology, National Key Research and Development Program of China (No.2021YFB2910109) and National Natural Science Foundation of China (No.62202465). Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of any finding agencies.

References

[1] Yehuda Afek, Anat Bremler-Barr, and Lior Shafir. 2017. Network anti-spoofing with SDN data plane. In *IEEE INFOCOM 2017-IEEE conference on computer communications*. IEEE, 1–9.

[2] Mazhar Javed Awan, Umar Farooq, Hafiz Muhammad Aqeel Babar, Awais Yasin, Haitham Nobanee, Muzammil Hussain, Owais Hakeem, and Azlan Mohd Zain. 2021. Real-time DDoS attack detection system using big data approach. *Sustainability* 13, 19 (2021), 10743.

[3] Afsaneh Banitalebi Dehkordi, MohammadReza Soltanaghaei, and Farsad Zamani Boroujeni. 2021. The DDoS attacks detection through machine learning and statistical methods in SDN. *The Journal of Supercomputing* 77, 3 (2021), 2383–2415.

[4] Narmeen Zakaria Bawany, Jawwad A Shamsi, and Khaled Salah. 2017. DDoS attack detection and mitigation using SDN: methods, practices, and solutions. *Arabian Journal for Science and Engineering* 42 (2017), 425–441.

[5] Roberto Bifulco and Gábor Rétvári. 2018. A survey on the programmable data plane: Abstractions, architectures, and open problems. In *2018 IEEE 19th International Conference on High Performance Switching and Routing (HPSR)*. IEEE, 1–7.

[6] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, et al. 2014. P4: Programming protocol-independent packet processors. *ACM SIGCOMM Computer Communication Review* 44, 3 (2014), 87–95.

[7] Nurefşan Sertbaş Bülbül and Mathias Fischer. 2020. SDN/NFV-based DDoS mitigation via pushback. In *ICC 2020-2020 IEEE international conference on communications (ICC)*. IEEE, 1–6.

[8] Coralie Busse-Grawitz, Roland Meier, Alexander Dietmüller, Tobias Bühler, and Laurent Vanbever. 2019. pforest: In-network inference with random forests. *arXiv preprint arXiv:1909.05680* (2019).

[9] Xiao-Fan Chen and Shun-Zheng Yu. 2016. CIPA: A collaborative intrusion prevention architecture for programmable network and SDN. *Computers & Security* 58 (2016), 1–19.

[10] Bruno Coelho and Alberto Schaeffer-Filho. 2022. BACKORDERS: using random forests to detect DDoS attacks in programmable data planes. In *Proceedings of the 5th International Workshop on P4 in Europe*. 1–7.

[11] Babu R Dawadi, Bibek Adhikari, and Devesh Kumar Srivastava. 2023. Deep learning technique-enabled web application firewall for the detection of web attacks. *Sensors* 23, 4 (2023), 2073.

[12] Ozgur Depren, Murat Topallar, Emin Anarim, and M Kemal Ciliz. 2005. An intelligent intrusion detection system (IDS) for anomaly and misuse detection in computer networks. *Expert systems with Applications* 29, 4 (2005), 713–722.

[13] Rohan Doshi, Noah Apthorpe, and Nick Feamster. 2018. Machine learning ddos detection for consumer internet of things devices. In *2018 IEEE Security and Privacy Workshops (SPW)*. IEEE, 29–35.

[14] Jon Dugan, Seth Elliott, Bruce A. Mah, Jeff Poskanzer, and Kaustubh Prabhu. 2014. iperf: A TCP, UDP, and SCTP Network Bandwidth Measurement Tool. https://iperf.fr/. Accessed: April 1, 2025.

[15] Lubna Fayez Eliyan and Roberto Di Pietro. 2021. DoS and DDoS attacks in Software Defined Networks: A survey of existing solutions and research challenges. *Future Generation Computer Systems* 122 (2021), 149–171.

[16] Seyed K Fayaz, Yoshiaki Tobioka, Vyas Sekar, and Michael Bailey. 2015. Bohatei: Flexible and elastic {DDoS} defense. In *24th USENIX security symposium (USENIX Security 15)*. 817–832.

[17] Sam Gao, Mark Handley, and Stefano Vissicchio. 2021. Stats 101 in p4: towards in-switch anomaly detection. In *Proceedings of the 20th ACM Workshop on Hot Topics in Networks*. 84–90.

[18] Feng Guo, Shidong Sun, Junjie Hu, Ning Zhang, and Zhiqiang Lv. 2024. CIPO: Efficient, lightweight and programmable packet scheduling. *Computer Networks* 245 (2024), 110355.

[19] Dingwen Hu, Peilin Hong, and Yixin Chen. 2017. FADM: DDoS flooding attack detection and mitigation system in software-defined networking. In *GLOBECOM 2017-2017 IEEE Global Communications Conference*. IEEE, 1–7.

[20] Joel Hypolite, John Sonchack, Shlomo Hershkop, Nathan Dautenhahn, André DeHon, and Jonathan M Smith. 2020. DeepMatch: Practical deep packet inspection in the data plane using network processors. In *Proceedings of the 16th International Conference on emerging Networking EXperiments and Technologies*. 336–350.

[21] John Ioannidis and Steven M. Bellovin. 2002. Implementing Pushback: Router-Based Defense Against DDoS Attacks. In *Network and Distributed System Security Symposium*.

[22] Samuel Jero, William Koch, Richard Skowyra, Hamed Okhravi, Cristina Nita-Rotaru, and David Bigelow. 2017. Identifier binding attacks and defenses in {Software-Defined} networks. In *26th USENIX Security Symposium (USENIX Security 17)*. 415–432.

[23] Min Suk Kang, Virgil D Gligor, Vyas Sekar, et al. 2016. SPIFFY: Inducing Cost-Detectability Tradeoffs for Persistent Link-Flooding Attacks.. In *NDSS*, Vol. 1. 53–55.

[24] Min Suk Kang, Soo Bum Lee, and Virgil D Gligor. 2013. The crossfire attack. In *2013 IEEE symposium on security and privacy*. IEEE, 127–141.

[25] Fatima Khashab, Joanna Moubarak, Antoine Feghali, and Carole Bassil. 2021. DDoS attack detection and mitigation in SDN using machine learning. In *2021 IEEE 7th International Conference on Network Softwarization (NetSoft)*. IEEE, 395–401.

[26] Changhoon Kim, Anirudh Sivaraman, Naga Katta, Antonin Bas, Advait Dixit, Lawrence J Wobker, et al. 2015. In-band network telemetry via programmable dataplanes. In *ACM SIGCOMM*, Vol. 15. 1–2.

[27] Jong-Hyouk Lee and Kamal Singh. 2020. Switchtree: in-network computing and traffic analyses with random forests. *Neural Computing and Applications* (2020), 1–12.

[28] Qing Li, He Huang, Ruoyu Li, Jianhui Lv, Zhenhui Yuan, Lianbo Ma, Yi Han, and Yong Jiang. 2023. A comprehensive survey on DDoS defense systems: New trends and challenges. *Computer Networks* (2023), 109895.

[29] Jiaye Lin, Qing Li, Guorui Xie, Yong Jiang, Zhenhui Yuan, Changlin Jiang, and Yuan Yang. 2023. In-Forest: Distributed In-Network Classification with Ensemble Models. In *2023 IEEE 31st International Conference on Network Protocols (ICNP)*. IEEE, 1–12.

[30] Jed Liu, William Hallahan, Cole Schlesinger, Milad Sharif, Jeongkeun Lee, Robert Soulé, Han Wang, Călin Caşcaval, Nick McKeown, and Nate Foster. 2018. P4v: Practical verification for programmable data planes. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on data communication*. 490–503.

[31] Zaoxing Liu, Hun Namkung, Georgios Nikolaidis, Jeongkeun Lee, Changhoon Kim, Xin Jin, Vladimir Braverman, Minlan Yu, and Vyas Sekar. 2021. Jaqen: A {High-Performance}{Switch-Native} approach for detecting and mitigating volumetric {DDoS} attacks with programmable switches. In *30th USENIX Security Symposium (USENIX Security 21)*. 3829–3846.

[32] Ajay Mahimkar, Jasraj Dange, Vitaly Shmatikov, Harrick M Vin, and Yin Zhang. 2007. dFence: Transparent Network-based Denial of Service Mitigation.. In *NSDI*,

Vol. 7. 327–340.
[33] Albert Mestres, Alberto Rodriguez-Natal, Josep Carner, Pere Barlet-Ros, Eduard Alarcón, Marc Solé, Victor Muntés-Mulero, David Meyer, Sharon Barkai, Mike J Hibbett, et al. 2017. Knowledge-defined networking. *ACM SIGCOMM Computer Communication Review* 47, 3 (2017), 2–10.
[34] Yu Mi and An Wang. 2019. Ml-pushback: Machine learning based pushback defense against ddos. In *Proceedings of the 15th International Conference on emerging Networking EXperiments and Technologies*. 80–81.
[35] Oliver Michel, Roberto Bifulco, Gabor Retvari, and Stefan Schmid. 2021. The programmable data plane: Abstractions, architectures, algorithms, and applications. *ACM Computing Surveys (CSUR)* 54, 4 (2021), 1–36.
[36] Meenakshi Mittal, Krishan Kumar, and Sunny Behal. 2023. Deep learning approaches for detecting DDoS attacks: A systematic review. *Soft computing* 27, 18 (2023), 13039–13075.
[37] Nour Moustafa and Jill Slay. 2015. UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In *2015 military communications and information systems conference (MilCIS)*. IEEE, 1–6.
[38] Mohammad Najafimehr, Sajjad Zarifzadeh, and Seyedakbar Mostafavi. 2023. DDoS attacks and machine-learning-based detection methods: A survey and taxonomy. *Engineering Reports* 5, 12 (2023), e12697.
[39] Euclides Carlos Pinto Neto, Sajjad Dadkhah, Raphael Ferreira, Alireza Zohourian, Rongxing Lu, and Ali A Ghorbani. 2023. CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment. *Sensors* 23, 13 (2023), 5941.
[40] Quamar Niyaz, Weiqing Sun, and Ahmad Y Javaid. 2016. A deep learning based DDoS detection system in software-defined networking (SDN). *arXiv preprint arXiv:1611.07400* (2016).
[41] Matheus P Novaes, Luiz F Carvalho, Jaime Lloret, and Mario Lemes Proença Jr. 2021. Adversarial Deep Learning approach detection and defense against DDoS attacks in SDN environments. *Future Generation Computer Systems* 125 (2021), 156–167.
[42] P4 Language Consortium. 2024. P4 Language. https://p4.org. Accessed: November 14, 2024.
[43] P4 language Organization. 2020. Performance of BMv2. https://github.com/p4lang/behavioral-model/blob/main/docs/performance.md. Accessed: November 14, 2024.
[44] Tian Pan, Nianbing Yu, Chenhao Jia, Jianwen Pi, Liang Xu, Yisong Qiao, Zhiguo Li, Kun Liu, Jie Lu, Jianyuan Lu, et al. 2021. Sailfish: Accelerating cloud-scale multi-tenant multi-service gateways with programmable switches. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*. 194–206.
[45] Bhavika Pande, Gargi Bhagat, Shanu Priya, and Himanshu Agrawal. 2018. Detection and mitigation of DDoS in SDN. In *2018 Eleventh International Conference on Contemporary Computing (IC3)*. IEEE, 1–3.
[46] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. 2011. Scikit-learn: Machine learning in Python. *the Journal of machine Learning research* 12 (2011), 2825–2830.
[47] Jesus Arturo Perez-Diaz, Ismael Amezcua Valdovinos, Kim-Kwang Raymond Choo, and Dakai Zhu. 2020. A flexible SDN-based architecture for identifying and mitigating low-rate DDoS attacks using machine learning. *IEEE Access* 8 (2020), 155859–155872.
[48] Trung V Phan, Nguyen Khac Bao, and Minho Park. 2016. A novel hybrid flow-based handler with DDoS attacks in software-defined networking. In *2016 Intl IEEE conferences on ubiquitous intelligence & computing, advanced and trusted computing, scalable computing and communications, cloud and big data computing, internet of people, and smart world congress (UIC/ATC/ScalCom/CBDCom/IoP/SmartWorld)*. IEEE, 350–357.
[49] Libardo Andrey Quintero González, Lucas Castanheira, Jonatas A Marques, Alberto E Schaeffer-Filho, and Luciano Paschoal Gaspary. 2023. Bungee-ml: A cross-plane approach for a collaborative defense against ddos attacks. *Journal of Network and Systems Management* 31, 4 (2023), 77.
[50] Christian Rossow. 2014. Amplification Hell: Revisiting Network Protocols for DDoS Abuse.. In *NDSS*. 1–15.
[51] Salvatore Sanfilippo. 2012. hping3: A Network Tool for TCP/IP Protocol Analysis. https://www.hping.org/. Accessed: April 1, 2025.
[52] Reneilson Santos, Danilo Souza, Walter Santo, Admilson Ribeiro, and Edward Moreno. 2020. Machine learning algorithms to detect DDoS attacks in SDN. *Concurrency and Computation: Practice and Experience* 32, 16 (2020), e5402.
[53] Maria Jose Erquiaga Sebastian Garcia, Agustin Parmisano. 2020. A labeled dataset with malicious and benign iot network traffic. *Stratosphere Laboratory: Praha, Czech Republic* (2020).
[54] Iman Sharafaldin, Arash Habibi Lashkari, Ali A Ghorbani, et al. 2018. Toward generating a new intrusion detection dataset and intrusion traffic characterization. *ICISSp* 1, 2018 (2018), 108–116.
[55] Iman Sharafaldin, Arash Habibi Lashkari, Saqib Hakak, and Ali A Ghorbani. 2019. Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy. In *2019 international carnahan conference on security technology (ICCST)*. IEEE, 1–8.
[56] Hisham Siddique, Miguel Neves, Carson Kuzniar, and Israat Haque. 2021. Towards network-accelerated ML-based distributed computer vision systems. In *2021 IEEE 27th International Conference on Parallel and Distributed Systems (ICPADS)*. IEEE, 122–129.
[57] Marek Šimon, Ladislav Huraj, and Michal Čerňanskỳ. 2015. Performance evaluations of IPTables firewall solutions under DDoS attacks. *Journal of Applied Mathematics, Statistics and Informatics* 11, 2 (2015), 35–45.
[58] Jagdeep Singh and Sunny Behal. 2020. Detection and mitigation of DDoS attacks in SDN: A comprehensive review, research challenges and future directions. *Computer Science Review* 37 (2020), 100279.
[59] Tushar Swamy, Alexander Rucker, Muhammad Shahbaz, Ishan Gaur, and Kunle Olukotun. 2022. Taurus: a data plane architecture for per-packet ML. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 1099–1114.
[60] Intel Tofino. 2020. P4-programmable ethernet switch ASIC that delivers better performance at lower power.
[61] Josy Elsa Varghese and Balachandra Muniyal. 2021. An efficient IDS framework for DDoS attacks in SDN environment. *IEEE Access* 9 (2021), 69680–69699.
[62] An Wang, Wentao Chang, Songqing Chen, and Aziz Mohaisen. 2018. Delving into internet DDoS attacks by botnets: characterization and analysis. *IEEE/ACM Transactions on Networking* 26, 6 (2018), 2843–2855.
[63] Guorui Xie, Qing Li, Chupeng Cui, Peican Zhu, Dan Zhao, Wanxin Shi, Zhuyun Qi, Yong Jiang, and Xi Xiao. 2022. Soter: Deep learning enhanced in-network attack detection based on programmable switches. In *2022 41st International Symposium on Reliable Distributed Systems (SRDS)*. IEEE, 225–236.
[64] Guorui Xie, Qing Li, Yutao Dong, Guanglin Duan, Yong Jiang, and Jingpu Duan. 2022. Mousika: Enable general in-network intelligence in programmable switches by knowledge distillation. In *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 1938–1947.
[65] Guorui Xie, Qing Li, Jiaye Lin, Gianni Antichi, Dan Zhao, Zhenhui Yuan, Ruoyu Li, and Yong Jiang. 2023. Dryad: Deploying Adaptive Trees on Programmable Switches for Networking Classification. In *2023 IEEE 31st International Conference on Network Protocols (ICNP)*. IEEE, 1–11.
[66] Jiarong Xing, Wenqing Wu, and Ang Chen. 2021. Ripple: A programmable, decentralized {Link-Flooding} defense against adaptive adversaries. In *30th USENIX Security Symposium (USENIX Security 21)*. 3865–3881.
[67] Yang Xu and Yong Liu. 2016. DDoS attack detection under SDN context. In *IEEE INFOCOM 2016-the 35th annual IEEE international conference on computer communications*. IEEE, 1–9.
[68] Qiao Yan, F Richard Yu, Qingxiang Gong, and Jianqiang Li. 2015. Software-defined networking (SDN) and distributed denial of service (DDoS) attacks in cloud computing environments: A survey, some research issues, and challenges. *IEEE communications surveys & tutorials* 18, 1 (2015), 602–622.
[69] Xiaoyong Yuan, Chuanhuang Li, and Xiaolin Li. 2017. DeepDefense: identifying DDoS attack via deep learning. In *2017 IEEE international conference on smart computing (SMARTCOMP)*. IEEE, 1–8.
[70] Noe Marcelo Yungaicela-Naula, Cesar Vargas-Rosales, and Jesus Arturo Perez-Diaz. 2021. SDN-based architecture for transport and application layer DDoS attack detection by using machine and deep learning. *IEEE Access* 9 (2021), 108495–108512.
[71] Marwane Zekri, Said El Kafhali, Noureddine Aboutabit, and Youssef Saadi. 2017. DDoS attack detection using machine learning techniques in cloud computing environments. In *2017 3rd international conference of cloud computing technologies and applications (CloudTech)*. IEEE, 1–7.
[72] Menghao Zhang, Guanyu Li, Shicheng Wang, Chang Liu, Ang Chen, Hongxin Hu, Guofei Gu, Qianqian Li, Mingwei Xu, and Jianping Wu. 2020. Poseidon: Mitigating volumetric ddos attacks with programmable switches. In *the 27th Network and Distributed System Security Symposium (NDSS 2020)*.
[73] Xiaoquan Zhang, Lin Cui, Fung Po Tso, and Weijia Jia. 2021. pHeavy: Predicting heavy flows in the programmable data plane. *IEEE Transactions on Network and Service Management* 18, 4 (2021), 4353–4364.
[74] Changgang Zheng, Zhaoqi Xiong, Thanh T Bui, Siim Kaupmees, Riyad Bensoussane, Antoine Bernabeu, Shay Vargaftik, Yaniv Ben-Itzhak, and Noa Zilberman. 2022. IIsy: Practical in-network classification. *arXiv preprint arXiv:2205.08243* (2022).
[75] Changgang Zheng, Zhaoqi Xiong, Thanh T Bui, Siim Kaupmees, Riyad Bensoussane, Antoine Bernabeu, Shay Vargaftik, Yaniv Ben-Itzhak, and Noa Zilberman. 2024. IIsy: Hybrid In-Network Classification Using Programmable Switches. *IEEE/ACM Transactions on Networking* (2024).
[76] Changgang Zheng, Mingyuan Zang, Xinpeng Hong, Liam Perreault, Riyad Bensoussane, Shay Vargaftik, Yaniv Ben-Itzhak, and Noa Zilberman. 2024. Planter: Rapid prototyping of in-network machine learning inference. *ACM SIGCOMM Computer Communication Review* 54, 1 (2024), 2–21.
[77] Guangmeng Zhou, Zhuotao Liu, Chuanpu Fu, Qi Li, and Ke Xu. 2023. An efficient design of intelligent network data plane. In *32nd USENIX Security Symposium (USENIX Security 23)*. 6203–6220.